

M1.1: MySQL vs PostgreSQL (关系型数据抉择)

Theory: MySQL 采用索引组织表且多用 B+ 树二级索引回表, MVCC 基于 Undo Log 链; PostgreSQL 使用堆表, MVCC 通过在行头存储 xmin/xmax 并在共享缓冲区中生成新版本行 (页内写多)。

Details: MySQL InnoDB 通过隔离级别 (如可重复读下的 Gap Lock) 防止幻读, 支持快照读和当前读; PostgreSQL 使用多版本并发控制与 SSI (基于 SIREAD 锁追踪的冲突依赖环检测), 依靠 VACUUM 清理死元组。

Trade-off: MySQL 对频繁写入与二级索引优化较好但复杂查询慢; PostgreSQL 支持丰富类型与复杂查询但有 WAL 与 Table Bloat 膨胀风险, 需定期 Auto-vacuum。

M1.4: InfluxDB vs TimescaleDB (时序数据库抉择)

Theory: InfluxDB 是专为指标优化的自研 TSM/TSI 引擎时序库; TimescaleDB 是基于 PostgreSQL 的 Hypertable 关系型扩展时序库。

Details: InfluxDB 通过 WAL 和 TSM 文件将时间序列数据列式存储, 使用 TSI 哈希表维护 Tag 索引; TimescaleDB 将大表自动按时间和分区列拆分为 Chunk, 继承 Postgres 的 B-Tree 索引和完整 SQL 生态。

Trade-off: InfluxDB 写吞吐高、存储压缩率极佳, 但高基数 Tag 爆炸会导致内存溢出; TimescaleDB 支持复杂 JOIN 和 SQL, 但数据压缩开销较大且吞吐略低于 InfluxDB。

M2.1: Kafka vs Apache Pulsar (高性能事件流抉择)

Theory: Kafka 采用计算与存储合一的单文件追加写与 PageCache 零拷贝架构; Pulsar 采用计算与存储分离 (Broker 与 BookKeeper) 的三层物理部署架构。

Details: Kafka 分区物理绑定到指定 Broker 的 Segment 日志文件上; Pulsar 将分区抽象为分段 (Ledgers), 由 BookKeeper 分布式节点 (Bookies) 分散存储, Broker 仅作无状态分发。

Trade-off: Kafka 架构极简、吞吐率极高, 但分区数量上限受限于 Broker 文件句柄, 且扩容需重平衡移动数据; Pulsar 支持海量分区和即时水平扩容, 但系统链路深、运维成本极高。

M2.2: RocketMQ vs RabbitMQ (企业级队列抉择)

Theory: RocketMQ 是基于 CommitLog 单文件混写与高并发拉取模式的 Java 企业级消息队列; RabbitMQ 是基于 AMQP 协议和 Erlang 并发 Actor 模型的推模式消息队列。

Details: RocketMQ 所有 Topic 写入同一个 CommitLog, 通过 ConsumerQueue 二级索引拉取消息, 支持分布式事务消息; RabbitMQ 通过 Exchange 路由键转发消息至 Queue, 利用 Erlang 信道和内存队列向消费者 Push 数据。

Trade-off: RocketMQ 适合金融级事务、海量堆积与延迟队列, 但消费端基于拉模式有空轮询开销; RabbitMQ 实时延迟极低、路由灵活, 但堆积能力较弱且 Erlang 语言门槛较高。

M1.2: MongoDB vs Cassandra (NoSQL 抉择)

Theory: MongoDB 是强一致文档数据库, 使用 B-Tree 索引; Cassandra 是高可用宽列存储, 基于 Dynamo 架构的无中心哈希环与 SSTable, 采用最终一致性与 Gossip 协议。

Details: MongoDB 依靠 WiredTiger 引擎的 B-Tree 与锁机制, 采用 Replica Set 多数派协议; Cassandra 依靠客户端读写一致性级别 (Quorum = R + W > N) 控制一致性, 写入时追加 CommitLog 后写入 MemTable。

Trade-off: MongoDB 适合 Schema 多变且需要强一致读写的业务; Cassandra 写入吞吐极高且无单点故障, 但不支持 ACID 复杂事务, 且存在读放大 (Read Repair) 开销。

M1.5: RocksDB vs LevelDB (单机嵌入式存储抉择)

Theory: RocksDB 是基于 LSM-Tree 并针对多核 CPU、SSD 进行深度优化的嵌入式 KV 引擎; LevelDB 是经典轻量级单线程 LSM 存储引擎。

Details: RocksDB 引入 Column Families 隔离数据空间, 支持多线程并发 MemTable 写入和并发 Compaction; LevelDB 使用单一 SSTable 结构和单线程后台合并。

Trade-off: RocksDB 吞吐极高、配置参数极丰富, 但调优难度极大且写放大/空间放大显著; LevelDB 极其轻巧稳定, 适合嵌入式或资源受限环境。

M1.3: Redis vs Memcached (分布式缓存抉择)

Theory: Redis 是单线程事件循环的多数据结构存储, 支持持久化和复制; Memcached 是多线程 Slab Allocation 内存级 KV 存储, 不支持持久化。

Details: Redis 使用 epoll I/O 多路复用, 通过 RDB 快照/AOF 日志和哨兵/集群实现高可用; Memcached 使用 LRU 链表和 Slab 隔离物理内存, 避免内存碎片但限制了 Value 大小为 1MB。

Trade-off: Redis 功能强、支持丰富类型和持久化, 但单线程大 Key 阻塞会导致整体响应抖动; Memcached 适合简单且纯内存的极速 KV 读取, 但需在客户端实现一致性哈希分片。

M1.6: CockroachDB vs TiDB (分布式 NewSQL 抉择)

Theory: CockroachDB 采用多 Raft 组 (Range) 对等架构, 底座为基于 Pebble 的 KV 存储; TiDB 采用计算与存储分离架构, 计算层 TiDB 无状态, 存储层 TiKV/TiFlash 使用 Multi-Raft。

Details: CockroachDB 使用无共享对等网络, 通过 HLC 协调事务可见性, 依靠 2PC 与两阶段锁执行事务; TiDB 通过 PD (Placement Driver) 集中调度物理 Region, 使用 TSO 全局授时和 Percolator 分布式事务模型。

Trade-off: CockroachDB 在多地域多中心部署时容灾与本地读写支持较好, 但全局写延迟高; TiDB 生态成熟且支持混合 HTAP (OLTP+OLAP), 但架构较重、组件多, 维护成本高。

M2.3: Redis Streams vs MQTT (轻量/物联网队列抉择)

Theory: Redis Streams 是内存级自增只追加日志消息流; MQTT 是专为物联网无线传感器设计的轻量级发布订阅传输协议。

Details: Redis Streams 使用 Radix Tree 存储消息 ID, 支持 Consumer Groups and ACK 确认机制; MQTT 基于 TCP 长连接, 通过 Broker 分发主题, 支持 QoS 0/1/2 三种消息递交保证等级。

Trade-off: Redis Streams 适合微服务内部轻量级事件分发, 但受限于内存容量且不支持复杂协议扩展; MQTT 是海量物联网设备连接的首选, 但后台 Broker 的集群扩容与状态一致性维护成本高。

M2.4: Event Sourcing vs Event-Driven (事件流转抉择)

Theory: Event Sourcing 将应用的所有状态变更记录为一系列不可变的事件流，状态通过重播事件计算得出；EDA 则是子系统通过发布/订阅离散事件进行解耦协作。

Details: Event Sourcing 依靠 Event Store 存储全量历史变更，使用快照进行数据状态加速重建；EDA 使用常规中间件路由消息，消费完即物理删除，无需留存全部事件轨迹。

Trade-off: Event Sourcing 能实现完美的审计回溯与时间旅行，但数据量随时间膨胀严重且 Schema 演进复杂；EDA 设计灵活直观，但由于事件无序性或丢失可能引发数据不一致风险。

M3.1: RESTful HTTP/1.1 vs gRPC HTTP/2 (传输协议抉择)

Theory: RESTful 默认使用 HTTP/1.1 的文本 JSON 传输，通过短连接或 Keep-Alive 复用；gRPC 基于 HTTP/2，使用二进制 Protobuf 序列化，实现强类型契约长连接。

Details: RESTful 通过 URI 和 HTTP 动词表达语义，无内建 Schema 约束；gRPC 支持双向流控制，通过 '.proto'生成多语言桩代码，依靠 HTTP/2 多路复用彻底消除头部阻塞。

Trade-off: RESTful 对浏览器极其友好，生态开放易调试，但冗余标头大、序列化开销高；gRPC 在内部微服务网络性能极高，但对外部 Web 浏览器不直观且调试相对复杂。

M3.2: GraphQL vs RESTful API (网关 API 抉择)

Theory: GraphQL 是单端点的声明式查询语言，允许客户端指定所需字段；RESTful 是多端点面向资源的传统 API 设计模式。

Details: GraphQL 通过 Schema 定义数据图谱，客户端使用 Query DSL 单次请求获取关联对象；RESTful 对每个资源暴露独立端点，获取关联数据往往需要发起多次物理请求。

Trade-off: GraphQL 有效防止 Over-fetching 并减少网络开销，但后端解析复杂，易引发 N+1 查询问题；RESTful 简单稳定，容易在网关或 CDN 级别执行 HTTP 缓存优化。

M3.5: Dubbo vs gRPC (微服务 RPC 框架抉择)

Theory: Dubbo 是面向 Java 生态、提供完整服务治理与多协议支持的分布式 RPC 框架；gRPC 是 Google 开源的基于 Protobuf 契约的通用低层多语言 RPC 框架。

Details: Dubbo 支持 Zookeeper/Nacos 注册中心，内建动态代理、路由负载均衡与降级策略；gRPC 原生只提供底层的传输通道，服务治理需要配合 Istio 网格等外部系统。

Trade-off: Dubbo 对 Java 应用开箱即用，治理能力强，但多语言互通性弱；gRPC 跨语言能力极强、底层性能高，但微服务治理的附加组件配置较重。

M2.5: Push vs Pull Consumer (消费者驱动模式)

Theory: Push 模式由服务端主动向消费者推送消息，消费者处于被动接收状态；Pull 模式由消费者根据自身处理能力主动向服务端发起拉取请求。

Details: Push 模式通过网关维持长连接信道或 HTTP/2 Server Push 推送，实时性强但易打爆消费端缓冲；Pull 模式通过维持拉取长轮询来平衡低延迟与消费速率。

Trade-off: Push 模式延迟极低，但当消费者慢时需要服务端做复杂的流量控制与挂起；Pull 模式让消费者拥有主动权，易于批量合并拉取，但需要妥善处理无消息时的空轮询资源开销。

M3.3: WebSockets vs Server-Sent Events (实时推送抉择)

Theory: WebSockets 提供双向、全双工的长连接通信通道；Server-Sent Events 是基于常规 HTTP 的单向、从服务器到客户端的事件流推送。

Details: WebSockets 在 TCP 握手后升级协议，不再遵循常规 HTTP 报文；SSE 使用标准 HTTP，标头为 'text/event-stream'，支持自动断线重连。

Trade-off: WebSockets 适合实时协同、双向高频交互游戏，但穿透防火墙难且对网关层有负载要求；SSE 开销低、更符合常规 HTTP 生态，适合实时通知、大模型流式响应输出。

M3.6: Kong vs Envoy (API 网关选型)

Theory: Kong 是基于 OpenResty (Nginx + Lua) 构建的经典七层反向代理 API 网关；Envoy 是基于 C++ 开发的高性能、面向云原生的服务网络代理。

Details: Kong 使用 Lua 插件生态，通过关系型数据库或声明式 YAML 管理路由状态；Envoy 基于事件驱动和多线程设计，通过控制面动态下发 xDS 协议热更新配置。

Trade-off: Kong 在传统网关限流、鉴权插件拓展上成熟简易；Envoy 更适合作为 Sidecar 或 Service Mesh 边界网关，支持高度动态的配置热更，但运维和二开门槛较高。

M2.6: Partition-based vs Hash Ring (分片路由抉择)

Theory: Partition Sharding 将消息根据 Key 哈希路由到物理分区的固定段中；Hash Ring Sharding (一致性哈希环) 将消息与节点都映射到统一的虚拟环形哈希空间。

Details: Partition Sharding 分区数固定，动态增加分区需要对数据重新哈希和物理迁移；一致性哈希环通过在环上顺时针查找节点路由，支持节点扩容时仅漂移部分 Key。

Trade-off: Partition Sharding 在静态拓扑下路由速度快、顺序消费保证极强，但不利于频繁弹性伸缩；一致性哈希环弹性高但存在负载倾斜，必须引入虚拟节点。

M3.4: Protocol Buffers vs JSON vs Avro (数据编码抉择)

Theory: JSON 是人类可读的无模式自解释文本格式；Protobuf 是带有显式 Tag 编号的二进制紧凑编码；Avro 是结合动态 Schema 与纯二进制载荷的免代码生成序列化格式。

Details: JSON 在运行时需要昂贵的字符串扫描与解析；Protobuf 将字段名映射为整数 Tag 并使用 Varint 变长整型编码；Avro 在反序列化时必须读取或传递 Schema JSON 定义进行解析。

Trade-off: JSON 极易调试与跨语言交互，但体积大、解析慢；Protobuf 性能好、类型安全，但需维护契约文件；Avro 适合大数据海量存储，但较依赖生态上下文传输。

M4.1: Raft vs Paxos (强一致共识抉择)

Theory: Raft 是通过强领导者 (Leader) 模型进行多副本日志强行同步的共识算法；Paxos 是去中心化的、基于两阶段提案 (Prepare/Accept) 提案凝聚的一致性共识协议。

Details: Raft 将共识拆分为 Leader 选举、日志复制与安全保证，Leader 拥有最新且唯一的提交日志项；Multi-Paxos 支持多提案者并行提交，允许乱序确定 Log 位置，再通过 Commit 补偿一致。

Trade-off: Raft 直观易懂、工程实现难度较低，但强 Leader 会导致局部脑裂切换时的短时间写中断；Paxos 极其鲁棒且并未写入好，但工程实现难度极大，极难保证无死角正确性。

M4.2: etcd vs ZooKeeper (分布式协调引擎)

Theory: etcd 是基于 Go 与 Raft 的现代 KV 存储协调引擎，面向容器云；ZooKeeper 是基于 Java 与 ZAB 协议的经典树形节点分布式协调组件。

Details: etcd 提供 HTTP/gRPC API，使用 MVCC 存储历史版本，通过 Watch 机制监听版本变化；ZooKeeper 使用内存树结构，提供原生的 TCP 长连接会话与 Watcher 通知机制。

Trade-off: etcd 易于容器部署、性能好且天然适配 K8s，但大 KV 值存储开销大；ZooKeeper 连接管理极其强壮、生态库丰富，但 JVM 开销大，且 ZAB 协议选举切换时间较长。

M4.3: Strong Consistency vs Eventual (一致性折中)

Theory: 强一致性要求任何读取都能立刻返回最新的写入，系统表现得像单机一样；最终一致性允许系统状态暂时不一致，但承诺在没有新写入后数据最终收敛。

Details: 强一致性使用分布式锁、2PC、线性化 Read 或 Raft ReadIndex 校验；最终一致性使用 Quorum 读写、向量时钟或 CRDT 冲突解决机制。

Trade-off: 强一致性完全防止脏读和乱序，但导致写可用性降低（CAP 理论中放弃 A）并拉长网络延迟；最终一致性写可用性极高、延时低，但客户端需容忍读取过时数据。

M5.1: Vector Clocks vs Hybrid Logical Clocks (因果时钟)

Theory: Vector Clocks（向量时钟）利用节点 ID 数组跟踪事件的绝对偏序因果关系；HLC（混合逻辑时钟）将物理时钟与逻辑计数器结合，生成单调递增的物理近似时钟。

Details: Vector Clocks 需要每个消息携带所有节点的时钟版本，随着集群规模增大体积呈线性增长；HLC 只包含物理毫秒戳与逻辑序号，数据结构极其紧凑。

Trade-off: Vector Clocks 能准确捕获并发冲突，但开销随集群增长严重；HLC 能够生成紧凑的因果时间戳并接近物理现实，但无法区分在同一个物理毫秒和逻辑序号内的并发冲突。

M5.4: Serverless (FaaS) vs Containerized (K8s) (计算部署选型)

Theory: Serverless (FaaS) 采用按需触发、函数级的事件驱动执行模型，无状态且零常驻计算；Containerized (K8s) 提供常驻微服务容器实例，管理完整生命周期的微服务集群。

Details: FaaS 在请求到达时动态拉起冷启动实例，按执行时间和请求次数精确计费；K8s 通过常驻 Pod 池分发流量，根据 HPA 控制资源水位。

Trade-off: FaaS 在轻量任务、闲置率高及定时脚本下成本极低，但有冷启动延迟且状态保留难；K8s 适合稳定高并发、有状态服务，但需要预先支付常驻资源开销。

M4.4: Gossip Protocol vs Centralized (服务发现通信)

Theory: Gossip Protocol（闲话协议）通过节点间随机对等传播来实现去中心化的状态收敛；Centralized Registry（中心化注册表）依靠所有客户端定时与中心节点进行健康探活。

Details: Gossip 基于 UDP 广播或 TCP 随机对等交换，收敛时间与节点数对数相关；注册表客户端使用 Heartbeat 向服务端定时发送心跳并拉取全量实例表。

Trade-off: Gossip 无单点故障、极具弹性，但网络通信包冗余度高且状态收敛存在延迟；中心注册表一致性强、路由准确，但若中心端网络切分会导致全局调度瘫痪。

M5.2: Sync IPC vs Async Event-Driven (服务协作选型)

Theory: 同步 IPC 调用由请求方阻塞等待被调用方处理并返回结果；异步 EDA 调用由请求方投递消息至 MQ 后立即返回，消息由消费方异步处理。

Details: 同步 IPC 调用在客户端执行超时与连接池重试，易形成调用链级联雪崩；异步 EDA 调用利用 MQ 的队列解耦写入压力，通过 ACK 保证消息送达。

Trade-off: 同步 IPC 调试直观、开发逻辑符合直觉，但强耦合且可用性为各环节乘积；异步 EDA 极具弹性、天然消峰，但调试与事务保障极难。

M4.5: Redlock vs ZK Ephemeral Node (分布式锁抉择)

Theory: Redlock 是基于 Redis 多主节点的红锁算法，依靠时间戳判定锁定；ZooKeeper 锁依靠共识协议与临时节点的唯一性来独占控制权。

Details: Redlock 需要客户端在多数独立的 Redis 主节点上成功执行 'SET NX' 才能获取锁；ZooKeeper 锁依靠创建同名临时节点，失败者通过 Watch 监听前一个节点的删除事件实现排队。

Trade-off: Redlock 性能极高、对 Redis 依赖简单，但在 GC 暂停或网络时钟漂移时存在双重锁定风险；ZooKeeper 锁安全性极高且支持锁超时自动释放，但创建和删除节点会导致频繁的共识开销。

M5.3: Sidecar Service Mesh vs Proxyless (网络治理选型)

Theory: Sidecar 网络通过拦截应用的网络流量并将其交由旁路代理处理；Proxyless 通过集成 gRPC SDK 直接与服务发现控制面交互，应用自理网络逻辑。

Details: Sidecar 网络使用 iptables/eBPF 劫持套接字流，对应用完全透明；Proxyless gRPC 通过 gRPC 核心内置 of xDS 客户端解析来自控制面的路由规则。

Trade-off: Sidecar 支持任意语言且应用零侵入，但引入了两跳网络延迟与可观的内存开销；Proxyless 零网关开销、性能极佳，但限制在 gRPC 生态且需要应用频繁升级 SDK。

M5.5: Blue-Green vs Canary vs Rolling (发布策略抉择)

Theory: 蓝绿部署同时维护两个完整独立的生产环境（老版本和新版本），通过网关一键切换；金丝雀部署逐步将少量生产流量引入新版本，验证无误后再全量替换；滚动部署按批次替换旧实例。

Details: 蓝绿部署在网关级别执行域名解析或负载均衡指向的瞬间切流；金丝雀部署使用网关根据 Cookie 或特定比率进行流控转发；滚动部署通过 K8s ReplicaSet 的 maxSurge 和 maxUnavailable 控制更新容器数量。

Trade-off: 蓝绿部署最安全、支持一键回滚，但需要两倍的物理资源成本；金丝雀部署利于在真实生产环境小范围验证以降低事故概率，但需要微服务系统处理好新旧版本 API 兼容；滚动部署资源开销低但发布过程相对漫长。

Zone T1: Opinionated CLI & Specs

MySQL MVCC stack, PostgreSQL heap index, Consistent Hash
Ring SD, Kafka PageCache, RabbitMQ backpressure, gRPC
multiplexing, HLC Logical Clock

Zone T2: Core Trade-off Matrix

MySQL_Postgres_MVCC_locking,	Mon-
goDB_Cassandra_NoSQL_consensus,	Re-
dis_Memcached_caching_Slab,	In-
fluxDB_TimescaleDB_TSDB_cardinality,	
RocksDB_LSM_Column_Families,	
CockroachDB_TiDB_TSO_NewSQL,	
Kafka_Pulsar_BookKeeper_decoupled,	
Raft_Paxos_consensus_split	